
Senior QA Engineer Technical Assessment

Electro Pi - Multi-tenant Cloud POS & Inventory Platform

CANDIDATE	Abanoub Gallah
DATE	1 September 2026
CODE	https://github.com/AbanoubGallah/ElectroPi

Summary

Everything in this document is backed by working code in the accompanying repository —

<https://github.com/AbanoubGallah/ElectroPi>. Rather than describing a framework, I built it: a Playwright + TypeScript suite with a mock POS application bundled in, so a reviewer can clone the repo and run `npm ci && npx playwright test` with no environment to provision.

The suite as submitted: **17 automated tests (8 API, 9 UI), all passing**, plus a **Postman collection of 9 requests and 38 assertions** verified under Newman.

I also verified the tests are meaningful rather than merely green — see § Evidence at the end. In short: I broke the application in two ways (changed the success-toast wording; made the API return `201` while persisting nothing) and confirmed the relevant tests failed. I re-ran the suite with network latency raised 6× and it passed unchanged, which is the practical proof that no hardcoded waits are hiding in it.

Part 1: UI Test Automation & Architecture

1.1 Framework choice

I would choose **Playwright** (with TypeScript).

The choice follows from four properties of *this* product rather than from general preference. Electro Pi is (a) multi-tenant, (b) REST-API-driven with dynamic UI components, (c) cloud-based, and (d) commercially sensitive to accuracy and stability. Scored against those:

Criterion (weighted to this product)	Playwright	Cypress	Selenium
Auto-waiting / flake resistance out of the box	Strong — web-first assertions retry until timeout	Strong — retry-ability built in	Weak — explicit <code>WebDriverWait</code> everywhere, by hand
Native API testing in the same framework	Yes — <code>APIRequestContext</code> , first-class	Via <code>cy.request</code> , adequate	No — needs <code>RestAssured</code> /requests alongside
WebKit/Safari coverage (merchants run POS on iPads)	Yes — bundled WebKit	Experimental, incomplete	Yes, but needs <code>SafariDriver</code> + real macOS
True multi-tenant session testing (two merchants, one test)	Yes — multiple isolated <code>BrowserContext</code> s in one test	Hard — one browser, one origin at a time	Possible, but a second <code>WebDriver</code> session is slow and heavy
Parallel execution and CI sharding	Built in and free (<code>--shard</code>)	Parallelisation is a paid Cloud feature	Needs Grid infrastructure to maintain
Debuggability of a CI failure	Trace Viewer: DOM snapshots, network, console, replay	Good — video + time-travel in Cypress Cloud	Weak — a screenshot and a stack trace
Speed (typical suite wall-clock)	Fast — out-of-process, real parallel workers	Moderate	Slowest — HTTP-hop per command
Team adoption cost	Low — same TS the app team writes	Low	Moderate

The three arguments that actually decide it:

- 1. API-first setup makes UI tests stable and fast.** Most UI flakiness is not the UI — it is unreliable preconditions. Playwright lets a test authenticate and seed inventory over the REST API in the same file, with the same tooling, then exercise only the UI behaviour under test. In this repo that is the `inventoryAsAdmin` fixture: it obtains a token over HTTP, injects the session, and lands on the Inventory page — no UI login, no repeated 3-second cost per test. The same `APIRequestContext` then covers Part 2 entirely, so API and UI coverage share one framework, one CI configuration, and one report.
- 2. Multi-tenancy needs real browser isolation.** The most expensive possible bug on this platform is one merchant seeing another's data. Playwright's `BrowserContext` is a cheap, fully isolated browser profile, so a single test can hold two authenticated merchants side by side and assert isolation directly. Cypress's in-browser architecture makes that awkward; Selenium can do it, but at the cost of a second full `WebDriver` session.
- 3. Debuggability is a first-class requirement, and traces answer it.** The assessment asks for easy debugging. `trace: 'on-first-retry'` records a complete timeline — every DOM snapshot, network call and console message — so a CI failure is replayed locally with `npx playwright show-trace` instead of being reproduced

by guesswork. That single feature is worth more to a QA team's throughput than any syntax preference.

Where I would not choose Playwright — stated plainly, because a framework recommendation without limits is not a recommendation:

- **Selenium** if we needed a real-device cloud at scale (BrowserStack/Sauce against dozens of legacy browser/OS combinations), if the team were polyglot (Java/C#/Python engineers contributing to one suite), or if a corporate standard already mandated WebDriver. Selenium's ecosystem breadth and W3C standardisation are genuine advantages there.
- **Cypress** if the priority were maximum developer-experience for component-level and in-app testing, the team were already deep in it, and Safari coverage did not matter. Its interactive runner is still the best authoring loop available.

Neither exception applies here, so: Playwright.

1.2 Design pattern

Page Object Model, extended with Component Objects and Playwright Fixtures — arranged as six layers. A plain POM handles reusability but tends to grow god-objects and duplicated setup; adding component objects and fixtures fixes both without the onboarding cost of full Screenplay.

```
src/
  config/env.ts      Layer 0  environment + credentials (single source of truth)
  api/               Layer 1  service clients (AuthApi, InventoryApi)
  pages/             Layer 2  page objects (BasePage, LoginPage, InventoryPage)
    components/      component objects (ToastComponent, SideNavComponent)
  fixtures/          Layer 3  dependency injection + guaranteed teardown
  data/              Layer 4  test-data factories (unique by construction)
  tests/
    api/  ui/        Layer 5  specs – business intent only
```

Layer 0 — Configuration. Nothing outside `env.ts` reads `process.env`. One file resolves environment, base URLs, tenant, users and named timeouts, and it throws a readable error on a missing variable instead of failing later as an `undefined` selector. The same suite runs against local, staging and pre-prod by changing environment variables only.

Layer 1 — API service clients. Thin, typed wrappers over the REST endpoints. They return the raw response so tests can assert on status codes and error bodies — a client that throws on non-2xx cannot test negative cases. They serve double duty: the subject of the API tests, and the fast path for seeding UI test preconditions.

Layer 2 — Page and Component Objects. Three rules keep this layer honest:

- *Locators are declared, never resolved.* A Playwright `Locator` is a lazy query re-evaluated on each use, so nothing holds a stale element handle across a React re-render.
- *No business assertions inside page objects.* Page objects expose behaviour and state; tests own the expectations. This is what lets one page object serve a happy path and a validation test without accumulating conditionals. The single exception is `waitUntilReady()`, which is readiness, not an assertion.
- *Shared UI becomes a component object.* The toast host and the sidebar belong to the app shell, so they are components injected into `BasePage` rather than duplicated per page. When the nav is restyled, exactly one file changes.

Layer 3 — Fixtures (the layer most POM implementations are missing). A test declares what it needs and Playwright constructs it:

```
test('...', async ({ inventoryAsAdmin, adminToken, cleanup }) => { ... });
```

This buys three things that `beforeEach` chains cannot:

- **Guaranteed teardown.** The `cleanup` fixture deletes every item a test created *even when the test fails*, because a fixture always unwinds. A red run therefore never poisons the next one.
- **Cheap authentication.** `adminToken` is worker-scoped: the suite logs in once per worker process, not once per test.
- **Composability without inheritance.** Setup combinations are assembled by declaration rather than by a deepening hierarchy of base classes.

Layer 4 — Data factories. `anItem({ price: -5 })` returns a valid payload with one field overridden, so negative tests stay one line each. Identifiers are unique by construction (`MS-<random>`), which is what makes the suite safe to run in parallel and re-runnable without tripping duplicate-SKU conflicts.

Layer 5 — Specs read as business intent. The Part 1.3 flow below contains no selector, no URL and no timeout.

Why not Screenplay? Screenplay (Actors, Tasks, Questions, Abilities) is the better model for suites with many user roles composing many reusable tasks, and I would revisit it if Electro Pi grew to a dozen roles. Today its cost is concrete — a second vocabulary the app developers do not know, more indirection per test, slower onboarding — and Playwright fixtures already deliver the composability that usually motivates the move. I would rather have developers contributing to a POM they can read than a purer architecture they avoid.

How this structure produces the four required qualities:

Requirement	Mechanism
Scalability	Layer separation + tag-based execution (<code>@smoke</code> , <code>@regression</code> , <code>@api</code> , <code>@security</code>) + CI sharding; adding a module means adding one page object and one spec
Reusability	Component objects for shared UI, service clients for setup, factories for data
Maintainability	A selector changes in exactly one place; typed contracts mean a backend rename breaks compilation, not a random assertion
Debuggability	Trace-on-retry, <code>test.step</code> narration matching the test case, a logger that annotates the HTML report, screenshots + video on failure

1.3 Implementation

The requested flow, from `tests/ui/inventory-create-product.spec.ts`. Each of the five steps is a `test.step`, so the HTML report reads like the test case it came from.

```
import { test, expect } from '../../src/fixtures/test-fixtures.js';
import { env } from '../../src/config/env.js';
import { aProductForm } from '../../src/data/products.js';
import type { InventoryItem } from '../../src/api/types.js';

test.describe('Inventory - create product', () => {
  /* This spec's subject IS the login screen, so it must not inherit a session. */
  test.use({ storageState: { cookies: [], origins: [] } });

  test('Store Admin can create a product and sees a success toast @smoke @regression',
```

```

async ({ loginPage, inventoryPage, cleanup }) => {
  const product = aProductForm();

  await test.step('Log in as a Store Admin', async () => {
    await loginPage.open();
    await loginPage.loginAs(env.users.storeAdmin);
  });

  await test.step("Navigate to the 'Inventory' module", async () => {
    await inventoryPage.nav.goToInventory();
    // Readiness gate: spinner gone + "Add product" enabled (both fetches done).
    await inventoryPage.waitForReady();
  });

  await test.step("Fill in the 'Product Name' and 'Price'", async () => {
    await inventoryPage.openNewProductForm();
    await inventoryPage.fillProductForm({ name: product.name, price: product.price });
  });

  const response = await test.step("Click 'Save'", async () => {
    return inventoryPage.save();
  });

  await test.step('Assert that a success toast message appears', async () => {
    await inventoryPage.toast.expectSuccess(/saved successfully/i);
  });

  /* Beyond the brief, but this is what makes the test trustworthy: the toast
   is a UI promise; the 201 and the grid row are the evidence it was kept. */
  await test.step('Verify the API accepted it and the grid reflects it', async () => {
    expect(response.status()).toBe(201);
    const created = (await response.json()) as InventoryItem;
    cleanup.push(created.id);

    expect(created).toMatchObject({
      item_name: product.name,
      price: Number(product.price),
      tenant_id: env.tenantId,
      category_name: env.electronicsCategory.name,
    });
    await expect(inventoryPage.rowByName(product.name)).toBeVisible();
  });
});

```

One deliberate addition to the brief. The five requested steps end at the toast. I assert the `201` and the grid row as well, because a success toast is only a *claim*. I demonstrated the difference: with the API changed to return `201` and persist nothing, a toast-only test still passes, while this test fails (see § Evidence).

Supporting page-object code — the parts that carry the stability design:

```

// src/pages/InventoryPage.ts

/**
 * Readiness for an async-rendered grid, expressed as three observable facts:
 * spinner gone -> data arrived -> the primary action is enabled.
 *
 * The third condition is the one that matters. The app enables "Add product"
 * only after BOTH /inventory/items and /categories resolve, so waiting on it
 * removes the entire class of "clicked too early" failures - with no sleep,
 * and without coupling the test to a request count.
 */
override async waitForReady(): Promise<void> {
  await expect(this.loadingSpinner).toBeHidden({ timeout: env.timeouts.action });
  await expect(this.addProductButton).toBeEnabled({ timeout: env.timeouts.action });
}

/**
 * Clicks Save and waits for the request to settle.
 */

```

```

* `waitForResponse` is registered before the click so the listener cannot
* miss a fast response - a subtle ordering bug that shows up as a 1-in-20 flake.
*/
async save(): Promise<Response> {
  const responsePromise = this.page.waitForResponse(
    (response) =>
      response.url().includes('/api/v1/inventory/items') &&
      response.request().method() === 'POST',
    { timeout: env.timeouts.action },
  );
  await this.saveButton.click();
  const response = await responsePromise;
  return response;
}

```

```

// src/pages/components/ToastComponent.ts
//
// This toast auto-dismisses after 4 seconds. The helper waits for visibility
// and reads the text in the SAME web-first assertion - never `waitFor()` then
// `innerText()`, which is the classic race that makes toast assertions flaky.

async expectSuccess(text?: string | RegExp): Promise<void> {
  await expect(this.success, 'a success toast should be shown')
    .toBeVisible({ timeout: env.timeouts.toast });
  if (text !== undefined) {
    await expect(this.success).toContainText(text, { timeout: env.timeouts.toast });
  }
}

```

The repository also contains the negative UI coverage this flow needs to be real: missing price (inline error, modal stays open with input intact, no row added), negative price, duplicate SKU, unauthenticated access to `/inventory`, tenant-badge verification, and a reload test proving persistence.

1.4 Stability — dynamic elements and flake prevention, without sleeps

A hardcoded sleep is always wrong in both directions: too short and the test is flaky, too long and the suite is slow. Every wait should instead be a wait *for an observable condition*. My approach has five parts.

1. Wait on conditions, and prefer semantic readiness over technical readiness

Playwright's web-first assertions (`expect(locator).toBeVisible()` , `.toBeEnabled()` , `.toHaveText()`) poll until the condition holds or the timeout expires, and its actions auto-wait for an element to be attached, visible, stable, enabled and unobstructed before acting. That handles most dynamic rendering for free.

The part that is not free is deciding *what* "ready" means. I define readiness per page, in terms of what the user can actually do:

- **Loading spinners:** `await expect(spinner).toBeHidden()` — assert the spinner's absence, never a duration.
- **Async data render:** wait for the *consequence* of the data arriving. On the Inventory page, "Add product" is enabled only after both `GET /inventory/items` and `GET /categories` resolve, so `toBeEnabled()` on that button is a single condition that subsumes both requests.
- **Async-populated inputs:** the category `<select>` is disabled until its options land, so `toBeEnabled()` is the gate before `selectOption`.
- **Network delays:** await the specific response — `page.waitForResponse(...)` registered *before* the click, so a fast response cannot be missed.

- **I do not use `networkidle`** . A POS dashboard polls; "no requests for 500ms" may never be true. Playwright discourages it, and it is a technical proxy for a question the UI can answer directly.

2. Selectors that survive a component refactor

Priority order, enforced in review:

1. `data-testid` — an explicit contract with the front-end team. I would agree it as a definition-of-done item for new components; it is cheap for developers and removes the single largest source of UI flake.
2. Role and label (`getByRole` , `getByLabel`) — user-visible semantics, which also surface accessibility defects.
3. CSS structure — last resort.

Never XPath, never generated class names (`.css-1x3fh9`), and never index-based lookups. Rows are located by business key: `rowBySku('MS-001')` , not `.nth(0)` — with parallel workers writing to the same tenant, "the first row" is not a stable concept.

3. Test isolation — the flake cause that gets misdiagnosed as timing

Practice	Why
One fresh <code>BrowserContext</code> per test	No cookie/storage bleed between tests
Unique data per test (<code>MS-<random></code>)	Two parallel workers can never collide on a SKU
Teardown in a fixture, not at the end of the test body	Cleanup still runs when an assertion fails
No shared mutable state, no ordering dependencies	Any test can run alone, in any order, in any shard
Preconditions seeded over the API	Removes the slowest, least relevant UI steps from setup

4. Determinism where the app is not deterministic

- **Route interception** (`page.route`) to stub third-party calls and to force error states — a 500 from the inventory service, or a timeout — which are otherwise nearly impossible to reproduce on demand.
- **Clock control** (`page.clock`) for anything time-dependent, such as an end-of-day sales cutoff, instead of waiting for wall-clock time.
- **Pinned environment**: browsers come from the pinned Playwright version, and CI runs a fixed viewport, timezone and locale, so a currency or date assertion cannot fail because a runner is in a different region.

5. Treating flakiness as a defect with an owner

- `retries: 1` **in CI only**. Locally there are no retries, so flakiness surfaces while it is cheap to fix. A test that passes on retry is reported as *flaky*, not green — that report is the triage queue.
- A **nightly flake hunt** runs the suite `--repeat-each=5 --retries=0` . This is the only reliable way to find a 1-in-20 failure before a developer loses an hour to it.
- **Quarantine with an expiry**. A persistently flaky test is tagged `@quarantine` , excluded from the blocking gate, and given a ticket with an owner and a due date. Quarantine is a holding pattern, not a graveyard.
- **Trace on first retry** so root-causing does not require reproduction.

Symptom → cause → fix

Symptom	Actual cause	Fix
"Element not found" that passes locally	Assertion ran before async render	Semantic readiness gate (<code>toBeEnabled</code> on the action the data unlocks)
Click lands on the wrong element	Layout shifted as data arrived	Playwright's actionability checks + wait for the settled state
Toast assertion fails intermittently	Toast auto-dismissed between <code>waitFor</code> and <code>innerText</code>	Single web-first assertion covering visibility and text
Passes alone, fails in the suite	Shared data or leaked session state	Unique data per test + fresh context + fixture teardown
Fails only on the CI runner	Slower machine, different viewport/timezone	Condition-based waits, pinned viewport/TZ/locale, environment-driven timeouts
Fails only in a specific shard	Hidden ordering dependency	Remove shared state; every test creates its own preconditions

Evidence this works rather than merely reads well: I re-ran the smoke suite with the application's artificial network latency raised from 350 ms to 2000 ms ($\approx 6\times$) with no code change. Both tests passed. A suite with hidden timing assumptions cannot do that.

Part 2: API Testing Strategy

POST /api/v1/inventory/items — Authorization: Bearer <token>

```
{ "item_name": "Wireless Mouse", "sku": "MS-001", "quantity": 50,
  "price": 25.00, "category_id": 3 }
```

2.1 Test scenarios

Every scenario below is implemented in `tests/api/inventory-items.spec.ts` (8 tests, all passing). Each asserts three things, not one: the **status code** (the contract), the **response body** (what the client consumes), and the **side effect** (what the database now holds, via a follow-up read). A test that checks only the status code passes against an endpoint that returns `201` and stores nothing.

Positive

P1 — Valid payload, Store Admin token

- *Input:* the exact payload above, with a valid `Bearer` token for a `STORE_ADMIN` of the tenant, and `category_id: 3` existing for that tenant.
- *Expected status:* **201 Created**
- *Expected result:* body echoes `item_name`, `sku`, `quantity: 50`, `price: 25.00`, `category_id: 3`; adds a server-generated `id` (UUID), `created_at`, and `category_name: "Electronics"`; `tenant_id` equals the caller's tenant; a `Location` header points at the new resource. A subsequent `GET /api/v1/inventory/items/{id}` returns **200** with the same values — proving persistence, not just a response. Response within the 2 s SLA.

P2 — Boundary values accepted

- *Input:* `quantity: 0` (legitimate: pre-order or out-of-stock line), `price: 0.01` (smallest chargeable amount), `item_name` at exactly the 120-character limit.
- *Expected status:* **201 Created**
- *Expected result:* values stored exactly as sent — `quantity` is `0`, not coerced to `null` or rejected as falsy (a very common defect), `price` is `0.01` with no floating-point drift, and the 120-character name is not silently truncated. Boundaries are where validation code is thinnest, which is why this is a positive test rather than an afterthought.

Negative

N1 — Missing or invalid bearer token

- *Input:* the valid payload with (a) no `Authorization` header, then (b) a tampered/expired token.
- *Expected status:* **401 Unauthorized** in both cases.
- *Expected result:* error code `UNAUTHENTICATED` / `TOKEN_INVALID`; **no item is created**; the response leaks nothing about the tenant or the schema. This is the highest-priority negative test on a write endpoint: an authentication bypass here would let any caller write into any merchant's inventory.

N2 — Invalid field values (validation)

- *Input*: a table of one violation per row — `item_name` missing; `item_name` blank/whitespace; `sku` missing; `sku` containing illegal characters; `quantity`: -5 ; `quantity`: 2.5 ; `price`: -25.00 ; `price` as the string "25.00" ; `price`: 25.005 (3 decimals); `category_id`: 999999 (does not exist for this tenant).
- *Expected status*: **422 Unprocessable Entity** (400 is also defensible; what matters is that it is documented and consistent).
- *Expected result*: code `VALIDATION_ERROR` with **field-level detail** — `{ "field": "price", "issue": "MUST_BE_GTE_0" }` — so the client can highlight the offending input; nothing is persisted. The test asserts the specific field *and* reason, because a generic "expect 422" passes even when the API blames the wrong field, which is exactly how unhelpful error messages reach production. Negative `price` and negative `quantity` deserve particular attention on a POS: they corrupt stock valuation and can enable a refund exploit.

N3 — Duplicate SKU

- *Input*: the same `sku` posted twice within the same tenant.
- *Expected status*: **409 Conflict**
- *Expected result*: code `DUPLICATE_SKU` , message naming the SKU, and — the assertion that matters — a follow-up list query returns **exactly one** record, proving no phantom second row was written. SKU uniqueness must be scoped per tenant, not globally: two different merchants may both legitimately sell `MS-001` , and a global unique constraint would be a serious multi-tenant design bug.

Additional scenarios I would not ship without

The brief asks for three negatives; these three are why I would still block a release. On a multi-tenant write endpoint, authorisation and tenant scoping are not extra credit — they are the risk.

#	Scenario	Status	Expected result
N4	Cashier token (authenticated, not authorised)	403 Forbidden	Code <code>FORBIDDEN</code> ; 403, not 401 — confusing the two breaks client retry logic and masks genuine RBAC bugs
N5	Malformed JSON body (<code>{ "item_name": "Broken", }</code>)	400 Bad Request	Code <code>MALFORMED_JSON</code> , never a 500 — an unhandled parser exception is both a stability and an information-disclosure issue
N6	Cross-tenant read of an item created by another merchant	404 Not Found	404 rather than 403: a 403 confirms the record exists and leaks another merchant's data volume

Also on the plan, beyond a code assessment's scope: rate limiting (429), oversized payloads (413), unsupported media type (415), concurrent same-SKU creation (a race the unique constraint must win), idempotency of retries, and a contract test against the OpenAPI spec so a silent field rename fails CI.

2.2 Postman automation

The full collection is in `postman/` and runs headless under Newman — **9 requests, 38 assertions, 0 failures**, verified.

Dynamic token generation (authentication before the request runs)

The token is obtained by a **collection-level pre-request script**, so it runs before *every* request and no request ever carries a hand-pasted token. It also **caches** the token and refreshes only when it is missing or within 30 seconds of expiry — so a 40-request run performs one login, not forty.

```
/**
 * Collection-level pre-request script: dynamic token generation.
```

```

  */
  const SKEW_MS = 30 * 1000;
  const token = pm.environment.get('access_token');
  const expiresAt = Number(pm.environment.get('access_token_expires_at') || 0);
  const stillValid = Boolean(token) && Date.now() < expiresAt - SKEW_MS;

  if (stillValid) {
    console.log('[auth] reusing cached token');
  } else {
    pm.sendRequest({
      url: pm.environment.get('base_url') + '/api/v1/auth/login',
      method: 'POST',
      header: { 'Content-Type': 'application/json' },
      body: {
        mode: 'raw',
        raw: JSON.stringify({
          email: pm.environment.get('store_admin_email'),
          password: pm.environment.get('store_admin_password')
        })
      }
    }, function (err, res) {
      // Fail loudly: a silent auth failure would surface as a confusing wall
      // of 401s on the real assertions instead of one clear error.
      if (err) { throw new Error('Token request failed: ' + err); }
      if (res.code !== 200) { throw new Error('Login failed: ' + res.code + ' ' + res.text()); }

      const body = res.json();
      pm.environment.set('access_token', body.access_token);
      pm.environment.set('access_token_expires_at', Date.now() + body.expires_in * 1000);
      pm.environment.set('tenant_id', body.user.tenant_id);
      console.log('[auth] new token cached for ' + body.expires_in + 's');
    });
  }
}

```

Supporting decisions:

- **Collection-level Bearer auth** set once to `{{access_token}}`; individual requests inherit it. The 401 test overrides it with `No Auth` for that request only — the negative case is expressed as configuration, not as a copy of the request with the header deleted.
- **Credentials live in the environment, never in the collection.** The password is a `secret` -type variable so it is masked in the UI and excluded from exports; in CI it is injected from the secret store. `base_url` is an environment variable too, so the same collection runs against local, staging and pre-prod.
- **Unique data per run.** A request-level pre-request script generates `request_sku = 'MS-' + Date.now().toString(36)`, so the collection is re-runnable without tripping the 409 path by accident.
- **Self-contained ordering.** Create → read back → duplicate conflict → validation/auth negatives → delete. The teardown request removes what the run created, so state does not accumulate.
- **In CI:** `newman run ... --reporters cli,junit --reporter-junit-export reports/newman-junit.xml` publishes results into the pipeline's test report (see [.github/workflows/pr-checks.yml](#)). For a larger suite I would keep Postman as the collaborative/exploratory surface and treat the Playwright API project as the authoritative gate — one contract, two consumers, no drift.

Tests-tab assertions for a successful 201 and the returned sku

```

/**
 * Assertions for a successful 201 Created, and verification of the
 * `sku` returned in the response body.
 */
const expectedSku = pm.collectionVariables.get('request_sku');

pm.test('Status code is 201 Created', function () {
  pm.response.to.have.status(201);
});

```

```

// Parse once, AFTER the status check, so a 4xx/5xx body cannot throw an
// unhelpful JSON parse error and mask the real failure.
const body = pm.response.json();

pm.test('Response body returns the same SKU that was sent', function () {
  pm.expect(body).to.have.property('sku');
  pm.expect(body.sku).to.eql(expectedSku);
});

pm.test('Response echoes the submitted item exactly', function () {
  pm.expect(body.item_name).to.eql('Wireless Mouse');
  pm.expect(body.quantity).to.eql(50);
  pm.expect(body.price).to.eql(25);
  pm.expect(body.category_id).to.eql(3);
});

pm.test('Server assigned an id and a creation timestamp', function () {
  pm.expect(body.id).to.be.a('string').and.to.have.lengthOf(36);
  pm.expect(Date.parse(body.created_at)).to.not.be.NaN;
});

pm.test('Item is scoped to the authenticated tenant', function () {
  // Multi-tenant platform: the row must belong to the caller's store.
  pm.expect(body.tenant_id).to.eql(pm.environment.get('tenant_id'));
});

pm.test('Location header points at the created resource', function () {
  pm.expect(pm.response.headers.get('Location'))
    .to.include('/api/v1/inventory/items/' + body.id);
});

pm.test('Response matches the documented schema', function () {
  // Schema validation catches contract drift (a renamed or dropped field)
  // that value-by-value assertions would miss.
  pm.response.to.have.jsonSchema({
    type: 'object',
    required: ['id', 'item_name', 'sku', 'quantity', 'price',
      'category_id', 'tenant_id', 'created_at'],
    properties: {
      id: { type: 'string' },
      item_name: { type: 'string' },
      sku: { type: 'string' },
      quantity: { type: 'integer', minimum: 0 },
      price: { type: 'number', minimum: 0 },
      category_id: { type: 'integer' },
      category_name: { type: ['string', 'null'] },
      tenant_id: { type: 'string' },
      created_at: { type: 'string' }
    }
  });
});

pm.test('Responds within the 2s API SLA', function () {
  pm.expect(pm.response.responseTime).to.be.below(2000);
});

// Hand the id to the chained requests (duplicate-SKU check, then cleanup).
pm.collectionVariables.set('created_item_id', body.id);

```

Two details worth calling out. The `sku` is asserted against the variable that was **sent**, not against a hardcoded `"MS-001"` — a hardcoded literal would still pass if the API echoed a stale or default SKU. And `pm.response.json()` is called after the status assertion so that a `500` produces "expected 201, got 500" rather than a JSON parse error that hides it.

Part 3: CI/CD & Database Validation

3.1 Database testing

Full script with commentary: `sql/01_verify_item_in_electronics.sql`.

The brief names two tables — `products` and `categories` — and the request payload supplies the column names. The query below uses **only those**. I have not built a schema around them: the columns that commonly exist on a platform like this one, and the one-line change each would require, are listed at the end of this section as things to confirm rather than assumed as fact.

The query (PostgreSQL):

```
SELECT
  p.id,
  p.item_name,
  p.sku,
  p.quantity,
  p.price,
  p.category_id,
  c.name AS category_name
FROM products p
INNER JOIN categories c
  ON c.id = p.category_id
WHERE p.sku = :sku
  AND c.name = 'Electronics';
```

Three decisions in that query:

1. **INNER JOIN**, because the assertion is "this product *is* in Electronics". If the category reference does not resolve, the row must not come back.
2. **The filter is on `c.name`, not on `p.category_id = 3`**. The API was given `category_id: 3`, so asserting `category_id = 3` proves only that the value we sent came back to us. Joining to `categories` and checking the **name** proves the id actually resolves to Electronics in the categories table — which is what "correctly assigned" means. This is the difference between verifying the write and re-reading our own input.
3. **Parameters are bound** (`:sku`), never concatenated — injection-safe in test tooling, and the query plan is cached.

The version the automated test actually runs. The query above returns "no rows" for three different failures — the item was never created, it was created against the wrong category, or its `category_id` resolves to nothing — and "no rows" is a poor failure message. This version always returns exactly **one row of booleans**, so the report names the expectation that broke:

```
WITH expected AS (
  SELECT CAST(:sku AS TEXT) AS sku, 'Electronics' AS category_name,
    'Wireless Mouse' AS item_name, 50 AS quantity,
    CAST(25.00 AS NUMERIC(12,2)) AS price, 3 AS category_id
),
actual AS (
  -- LEFT JOIN here (unlike above) so a product whose category_id points at
  -- nothing still comes back, with category_name = NULL. That separates
  -- "orphaned reference" from "never inserted".
  SELECT p.id, p.item_name, p.sku, p.quantity, p.price, p.category_id,
    c.name AS category_name
  FROM products p
  LEFT JOIN categories c ON c.id = p.category_id
  WHERE p.sku = (SELECT sku FROM expected)
```

```

)
SELECT
  (SELECT COUNT(*) FROM actual) = 1 AS exactly_one_row,
  a.id IS NOT NULL AS product_exists,
  a.category_name IS NOT NULL AS category_reference_resolves,
  a.category_name = e.category_name AS assigned_to_electronics,
  a.category_id = e.category_id AS category_id_as_submitted,
  a.item_name = e.item_name AS item_name_matches,
  a.quantity = e.quantity AS quantity_matches,
  a.price = e.price AS price_matches_exactly,
  a.id, a.item_name, a.quantity, a.price, a.category_id, a.category_name
FROM expected e
LEFT JOIN actual a ON TRUE; -- expectations always present, so a missing
                             -- product yields false, not an empty result set

```

Validated, not just written. I ran this logic against four fixtures (SQLite harness in `sql/validate_logic.sh`, since the only dialect differences here are casting and how booleans render):

Fixture	exactly_one_row	product_exists	category_reference_resolves	assigned_to_electronics
Correct row	true	true	true	true
Wrong category (Groceries)	true	true	true	false
<code>category_id</code> resolves to nothing	true	true	false	NULL
Never created	false	false	false	NULL

Rows two and three are the point: both are failures, but the output says *which* — a wrong category assignment versus a broken reference. An `INNER JOIN` -only check reports both as "nothing found", indistinguishable from "the item was never created".

A third query in the same file sweeps for inventory rows whose `category_id` resolves to nothing at all (expected: zero rows). It runs nightly rather than per test, and it catches the class of corruption an API-level suite cannot see — because the API happily returns the id it was handed.

Extensions to confirm against the real schema

The queries use only documented columns. Three things are common on a platform like this, each needs a one-line change, and each is a real defect if it is present and ignored — so these are the first questions I would ask the backend team.

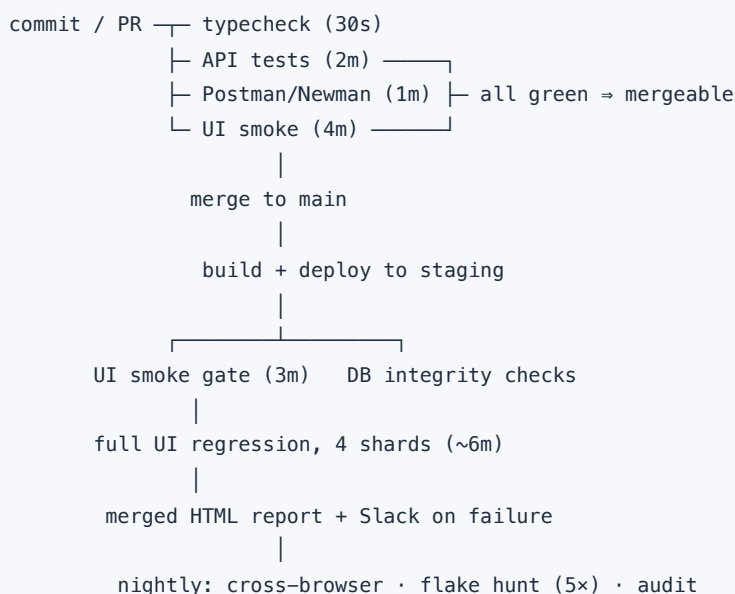
If the schema has	Change	Why it matters
A tenant column (<code>tenant_id</code> / <code>store_id</code>)	<code>ON c.id = p.category_id AND c.tenant_id = p.tenant_id</code> , plus <code>AND p.tenant_id = :tenant_id</code>	The platform is multi-tenant, so this is the first thing I would check. The join condition matters as much as the filter: if categories are tenant-scoped and the join is on <code>category_id</code> alone, it will attach another merchant's category row — a bug a single-tenant test dataset never reveals. And a verification query that reads a shared table untenanted can pass on another merchant's data. It also enables the isolation check worth having on any multi-tenant write path: the same SKU must return zero rows for any other tenant.
Soft deletes (<code>deleted_at</code> / <code>is_deleted</code>)	<code>AND p.deleted_at IS NULL</code>	Without it, a "found" row may be one the API already deleted; with it, a missing row is correctly reported as absent rather than as never created.
<code>price</code> stored as <code>FLOAT</code> / <code>DOUBLE</code>	<code>ABS(p.price - 25.00) < 0.005</code> instead of <code>=</code>	An exact comparison is only safe against <code>NUMERIC</code> / <code>DECIMAL</code> . On a float column, <code>25.00</code> can be stored as <code>24.999999999</code> , and the assertion then fails intermittently on a <i>correct</i> value. I would also raise storing money as <code>NUMERIC(12,2)</code> as a defect in its own right.

How this fits into the test, in practice. The database assertion runs *after* the API assertion and is scoped to data the test itself created. Rules I hold to: a **read-only** connection against a **read replica** — a test suite should never hold locks on the primary; credentials from the secret store; and queries that verify *what the API claims to have done* rather than replacing it. Tests that write directly to the database bypass business logic and rot into a second, contradictory implementation of it.

3.2 CI/CD integration

Three workflows in `.github/workflows/` , all valid and committed: `pr-checks.yml` , `post-deploy-e2e.yml` , `nightly.yml` . GitHub Actions here; the same structure maps directly onto GitLab CI stages or a Jenkins declarative pipeline.

The pipeline:



1. Containerised, reproducible runs. Tests execute in the official Playwright container (or with `npx playwright install --with-deps`), so browser versions are pinned to the Playwright version in `package-lock.json` . "It passed on my machine" stops being a category of failure.

2. Configuration and secrets. No credentials in the repo. `BASE_URL` and tenant come from environment variables; passwords and the read-only database URL come from GitHub Environment secrets, scoped per environment so a fork's PR cannot read staging credentials. The suite is environment-agnostic by construction — the same code runs locally against the bundled mock app and against staging.

3. Caching, deliberately keyed. `actions/setup-node` caches npm; browser binaries (~120 MB) are cached under a key derived from the resolved Playwright version, so the cache invalidates automatically on an upgrade instead of serving a stale browser. Worth about 40 seconds per job.

4. Parallelism and sharding. Playwright runs workers in parallel inside a job; `--shard=i/4` splits the regression suite across four runners, and `playwright merge-reports` combines the blob reports into **one** HTML report. Reviewers should not have to open four tabs. Wall-clock time is what decides whether a suite gets run or quietly skipped — 24 minutes becomes about 6.

5. Reporting and artifacts. JUnit XML feeds the platform's native test view; the HTML report, screenshots, videos and traces are uploaded on failure (`if: always()`) with retention tuned by value — 3 days for shard blobs, 30 days for the merged regression report. Every failing CI run therefore ships with a replayable trace, so triage does not start with "can you reproduce it?".

6. Quality gates. Branch protection requires typecheck, API tests, Newman and UI smoke to pass before merge. `forbidOnly: true` in CI fails the build if a `test.only` was left in a spec — otherwise CI silently runs one test and reports green. `fail-fast: false` across shards so one shard's failure does not hide the others' results.

7. Test data. Every test creates and cleans up its own data through fixtures; identifiers are unique by construction, so parallel jobs and repeated runs never collide. No shared golden dataset that drifts and no ordering dependency between tests.

8. Feedback loop. Failures post to the squad's Slack channel with a direct link to the run and the merged report. The nightly flake hunt (`--repeat-each=5 --retries=0`) keeps `retries: 1` honest by surfacing non-determinism as a tracked defect rather than letting a retry hide it.

Ownership. The suite lives in the repository it tests, is reviewed like application code, and developers are expected to add `data-testid` hooks as part of a component's definition of done. A QA framework maintained in a separate repo by one person is a framework that gets bypassed.

3.3 Execution strategy — what runs where, and why

The governing principle: **cost of a test should scale inversely with how often it runs**. API tests are cheap, fast and deterministic, so they run constantly; UI tests are expensive, slower and more fragile, so they run in tiers with the cheapest, highest-value slice first.

Stage	Trigger	What runs	Target time	Blocking?
1. Static analysis	Every push	Typecheck, lint	< 1 min	Yes
2. API tests	Every push / PR	Full API suite (<code>--project=api</code>) + Newman collection	2–3 min	Yes
3. UI smoke	Every PR	<code>@smoke</code> only — login, create product, tenant badge	3–5 min	Yes
4. Post-deploy smoke gate	After deploy to staging	<code>@smoke</code> against the deployed env	3 min	Yes — gates the regression run
5. UI regression	After the smoke gate passes	<code>@regression</code> , sharded 4x	~6 min	Yes for release promotion
6. DB integrity	After deploy	Referential integrity + tenant isolation sweeps	1 min	Yes
7. Nightly	Schedule (02:00 weekdays)	Cross-browser (WebKit for iPad terminals), flake hunt 5x, dependency audit	30–45 min	No — reports
8. Pre-release	Release branch / tag	Everything, plus performance and security scans	60 min	Yes

Why the full API suite on every commit, but only smoke UI. API tests are seconds each, have no rendering layer to be fragile about, and cover the business rules where the real risk lives on this platform — validation, RBAC, tenant scoping, duplicate SKUs. They are the highest-value-per-second tests we have, so there is no reason to sample them. UI tests are an order of magnitude slower and inherently more fragile; running 200 of them on every push buys little over the ~8 that would catch a genuinely broken build, and it trains the team to ignore CI. So: **all** the API tests, and the UI tests that would block a release.

Why UI regression runs only after deployment. A full UI regression is meaningful against a deployed, integrated environment — real network, real service dependencies, real data volumes. Running it against an ad-hoc PR environment costs more and tells us less. And it runs *behind* the smoke gate: if login is broken there is no value in spending 20 minutes discovering that in 200 other tests. Fail in 3 minutes and roll back.

Why cross-browser and flake detection are nightly. Both are valuable and neither is worth paying for on every commit. WebKit coverage matters here — merchants run POS terminals on iPads — but a WebKit-specific regression is rare enough that catching it within 24 hours is the right trade. Paying for it on every push is how a team ends up disabling CI.

The developer's experience of this, which is the actual point: a broken validation rule comes back in about 2 minutes. A broken login flow, about 5. A subtle rendering regression in a rarely used module, within the day. Fast feedback where it is cheap; thoroughness where it is warranted.

Evidence

Everything below was executed in preparing this submission; the commands are in the repository README.

Check	Result
API suite (<code>npx playwright test --project=api</code>)	8 passed in 4.7 s
UI suite (<code>npx playwright test --project=ui-chromium</code>)	9 passed in 13.6 s
Postman collection under Newman	9 requests, 38 assertions, 0 failures
TypeScript strict typecheck	Clean
SQL verification logic against 4 fixtures	Discriminates all 4 cases correctly
GitHub Actions workflows	3 workflows, valid YAML

Latency resilience (no hidden sleeps). Re-ran the smoke suite with the application's artificial latency raised from 350 ms to 2000 ms ($\approx 6\times$), no code change: **both tests passed**. Timing-dependent tests fail this.

Mutation testing (do the assertions actually bite?). A green suite proves nothing unless it can go red for the right reasons. I introduced two defects:

Injected defect	Expected	Observed
Success-toast wording changed (<code>"saved successfully"</code> $\rightarrow$ <code>"stored ok"</code>)	Smoke test fails	Failed — <code>toContainText</code> mismatch, with screenshot
API returns <code>201</code> but persists nothing	Persistence tests fail	Failed — both the API contract test (<code>TC-API-01</code>) and the UI reload test

The second mutation is the one I care about most: a toast-only test would have stayed green while the product silently lost every item a merchant created.

Assumptions

Stated because they shaped the code rather than being left implicit.

1. **No environment was provided**, so the repository includes a mock POS application (`mock-app/` , Node core modules only) implementing the documented endpoint contract, plus login, an async-rendered inventory grid, RBAC, multi-tenancy and transient toasts. It exists to make the framework runnable and to genuinely exercise the stability techniques described in §1.4 — the tests point at `BASE_URL` , so the identical suite runs against staging.
2. **Validation errors return 422** with a `{ error: { code, message, details } }` envelope. `400` is equally defensible; the tests assert the documented contract, whichever it is.
3. **The database schema was not provided**, so the SQL in §3.1 uses only the two tables the brief names and the columns the request payload supplies. It assumes nothing beyond that. Columns that commonly exist — a tenant column, soft deletes, a float `price` — are listed there as one-line extensions to confirm with the backend team, not baked into the shipped query.
4. **In the mock application** (not a claim about the real one) `sku` is unique **per tenant** rather than globally, which is the correct design for a multi-tenant catalogue: two merchants may both legitimately sell `MS-001` . `price` is treated as a 2-decimal value.
5. `category_id: 3` **resolves to Electronics**, per the assessment payload and the Part 3 requirement.